

R10 — Serial e telemetria

A placa passa a contar a sua história a um segundo computador, e a história vira dado

Microcontroladores — DENE/UFMT

OBJETIVOS

- Configurar a EUSART a partir do divisor calculado, e medir no osciloscópio a taxa que ele produz.
- Diagnosticar uma ligação serial pela aparência do defeito.
- Medir quanto a transmissão bloqueante tira do processador, e eliminar o bloqueio com fila e interrupção.
- Registrar em arquivo o degrau do aquecedor e o termostato em funcionamento, no formato que o R13 vai usar.
- Receber comandos pela serial e mudar o alvo com o sistema rodando.

CH4-6 (RS232_TX) e CH4-8 (RS232_RX)	ON — RC6 e RC7 ligados ao conector DB9 (CN4)	NESTE KIT
CH4-5 e CH4-7 (RS485)	OFF — disputam os mesmos pinos	
CH5 (relés)	Todas em OFF — os relés 1 e 2 estão ligados a RC6 e RC7	
CH1-7 (TEMP)	ON — LM35 em RA0/AN0	
CH1-1, CH1-5 e CH1-6	OFF — também chegam a RA0	
CH3-3 (HEATER)	ON a partir da Parte 3 — aquecedor em RC1	
CH3-6 (BUZZER)	ON só na Parte 2	
Demais chaves de CH3	OFF	

Cabo: DB9 do kit (CN4) ao computador, direto ou por adaptador USB-RS232. O cabo é de extensão, não *null-modem*. **No computador:** um terminal serial e Python 3 com pyserial (pip install pyserial).

A verificar antes da sessão: a porta que o adaptador cria (COMx ou /dev/ttyUSBx) em cada computador da bancada.

O aquecedor precisa estar frio no início da Parte 3. O degrau só vale se partir da temperatura ambiente. Não ligue CH3-3 antes dela.

ATENÇÃO

Pontuação. As seis tarefas somam 8,0, e as previsões P1 a P5 da aula 10, preenchidas antes da sessão, valem 2,0. A nota de cada tarefa e de cada previsão está na margem, ao lado dela. A seção *Se sobrar tempo*, no fim, não vale nota: é para quem terminar antes.

1 Previsões — entregues no início da sessão

As mesmas perguntas do fim da aula 10. Quem trouxe a folha preenchida copia aqui as respostas; a nota é do raciocínio, não do número.

P1. Com $n = 416$, que taxa real você espera medir? Qual é a largura de um bit?

PREVISÃO · 0,4 pt

P2. Com o terminal em 19200 em vez de 9600, o que se vê — nada, ou lixo?

PREVISÃO · 0,4 pt

P3. Com a transmissão bloqueante e o buzzer tocado pelo laço, o que se ouve?

PREVISÃO · 0,4 pt

P4. Com fila e interrupção, o que muda no som? E no pino TX?

PREVISÃO · 0,4 pt

P5. Quantas horas de telemetria a 1 Hz cabem num arquivo de 1 MB, no formato de texto da aula?

PREVISÃO · 0,4 pt

2 O programa

Um programa só. A cada segundo ele lê o LM35, decide o aquecedor e envia uma linha no formato da aula 10:

```
t_ms,temp_d,alvo_d,aquecedor,comutacoes
125000,398,400,1,17
```

Pela serial, ele aceita dois comandos, terminados em Enter: A450 muda o alvo para 45,0 °C, e D1 / D0 liga e desliga o **modo degrau**, em que o aquecedor fica ligado direto. O botão não entra: a interface agora é o computador.

```
#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>

#define BLOQUEANTE      1          /* Tarefa 3: 1 = espera TXIF; 0 = fila */
#define BUZZER          1          /* Parte 2: la tocado pelo laco */
#define PRECARGA_5MS    45536u     /* 5 ms a 250 ns (R7) */
#define PRECARGA_LA     60991u     /* meio periodo de 440 Hz (R7) */
#define H                5          /* histerese: 0,5 grau para cada lado */

/* ----- interrupcao */
#define FILA 64
static volatile uint8_t fila[FILA];
static volatile uint8_t entra = 0, sai = 0;

volatile uint8_t segundo = 0;      /* sinalizador de 1 s */
volatile char linha[16];           /* comando recebido */
volatile uint8_t nlin = 0, linha_pronta = 0;

void __interrupt() tratar(void)
{
    static uint8_t tiques = 0;

    if (INTCONbits.TMR0IF) {        /* 5 ms */
        INTCONbits.TMR0IF = 0;
        TMR0H = (uint8_t)(PRECARGA_5MS >> 8);
        TMR0L = (uint8_t)(PRECARGA_5MS & 0xFF);
        if (++tiques >= 200u) { tiques = 0; segundo = 1; }
    }

    if (PIE1bits.TXIE && PIR1bits.TXIF) { /* transmissor vazio */
        if (entra == sai) {
            PIE1bits.TXIE = 0;          /* nada a enviar: desliga a fonte */
        } else {
            TXREG = fila[sai];
            sai = (uint8_t)((sai + 1u) % FILA);
        }
    }

    if (PIR1bits.RCIF) {            /* chegou um byte */
```

```

        if (RCSTAbits.OERR) {                                /* estouro: a recepcao PAROU */
            RCSTAbits.CREN = 0;
            RCSTAbits.CREN = 1;
        }
        char c = RCREG;                                      /* ler RCREG limpa RCIF */
        if (c == '\n' || c == '\r') {
            linha[nlin] = '\0';
            nlin = 0;
            linha_pronta = 1;
        } else if (nlin < sizeof(linha) - 1u) {
            linha[nlin++] = c;
        }
    }
}

/* ----- serial */
static void uart_iniciar(void)
{
    TRISCbits.TRISC6 = 0; TRISCbits.TRISC7 = 1;
    BAUDCONbits.BRG16 = 1; TXSTAbits.BRGH = 1;
    SPBRGH = 0x__; SPBRG = 0x__; /* n = _____ -- Tarefa 1 */
    TXSTAbits.SYNC = 0;
    RCSTAbits.SPEN = 1; TXSTAbits.TXEN = 1; RCSTAbits.CREN = 1;
    PIE1bits.RCIE = 1;
}

static void uart_byte(uint8_t b)
{
    #if BLOQUEANTE
        while (!PIR1bits.TXIF) { } /* espera o transmissor */
        TXREG = b;
    #else
        uint8_t prox = (uint8_t)((entra + 1u) % FILA);
        while (prox == sai) { } /* fila cheia: raro */
        fila[entra] = b;
        entra = prox;
        PIE1bits.TXIE = 1; /* acorda o transmissor */
    #endif
}

static void enviar_u32(uint32_t v) /* sem printf */
{
    char buf[11];
    uint8_t i = 0;
    do { buf[i++] = (char)('0' + (uint8_t)(v % 10u)); v /= 10u; } while (v != 0u);
    while (i != 0u) { uart_byte((uint8_t)buf[--i]); }
}

/* ----- conversor (R5) */
static void adc_iniciar(void)
{
    TRISAbits.TRISA0 = 1;
    ADCON1 = 0x0E; /* so AN0 analogico */
    ADCON2 = 0b10010101; /* direita, 4 TAD, FOSC/16 */
    ADCON0 = 0x01; /* AN0, ligado */
}

static int16_t temperatura_decimos(void)

```

```

{
    ADCON0bits.GO = 1;
    while (ADCON0bits.GO) { }
    uint16_t c = ((uint16_t)ADRESH << 8) | ADRESL;
    return (int16_t)(((uint32_t)c * 625UL) >> 7);
}

/* ----- buzzer pelo laco */
static void buzzer_atender(void)
{
    #if BUZZER
        if (PIR1bits.TMR1IF) { /* consulta, de proposito */
            PIR1bits.TMR1IF = 0;
            TMR1H = (uint8_t)(PRECARGA_LA >> 8);
            TMR1L = (uint8_t)(PRECARGA_LA & 0xFF);
            LATCbits.LATC2 ^= 1;
        }
    #endif
}

void main(void)
{
    uint32_t t_ms = 0;
    int16_t temp, alvo = 400; /* 40,0 graus */
    uint8_t aquecedor = 0, degrau = 0;
    uint16_t comutacoes = 0;

    adc_iniciar(); /* ADCON1 = 0x0E: so AN0 analogico */
    LATCbits.LATC1 = 0; TRISCbits.TRISC1 = 0; /* aquecedor */
    LATCbits.LATC2 = 0; TRISCbits.TRISC2 = 0; /* buzzer */
    uart_iniciar();

    TMR0H = (uint8_t)(PRECARGA_5MS >> 8); TMR0L = (uint8_t)(PRECARGA_5MS & 0xFF);
    T0CON = 0x88; INTCONbits.TMR0IF = 0; INTCONbits.TMR0IE = 1;
    TMR1H = (uint8_t)(PRECARGA_LA >> 8); TMR1L = (uint8_t)(PRECARGA_LA & 0xFF);
    T1CON = 0x81; /* Timer1 sem interrupcao: o laco consulta */
    INTCONbits.PEIE = 1; INTCONbits.GIE = 1;

    for (;;) {
        buzzer_atender();

        if (linha_pronta) { /* comando: A450, D1, D0 */
            if (linha[0] == 'A') {
                int16_t v = 0;
                for (uint8_t i = 1; linha[i] >= '0' && linha[i] <= '9'; i++) {
                    v = (int16_t)(v * 10 + (linha[i] - '0'));
                }
                if (v >= 250 && v <= 600) { alvo = v; }
            } else if (linha[0] == 'D') {
                degrau = (linha[1] == '1');
            }
            linha_pronta = 0;
        }

        if (segundo) {
            segundo = 0;
            t_ms += 1000u;
            temp = temperatura_decimos();
        }
    }
}

```

```

uint8_t antes = aquecedor;
if (degrau)                { aquecedor = 1; }
else if (temp < alvo - H)   { aquecedor = 1; }
else if (temp > alvo + H)   { aquecedor = 0; }
if (aquecedor != antes)    { comutacoes++; }
LATCbits.LATC1 = aquecedor;

enviar_u32(t_ms);           uart_byte(',');
enviar_u32((uint32_t)temp);  uart_byte(',');
enviar_u32((uint32_t)alvo);  uart_byte(',');
uart_byte((uint8_t)('0' + aquecedor));  uart_byte(',');
enviar_u32(comutacoes);
uart_byte('\r');  uart_byte('\n');
    }
}
}

```

E, no computador, o registro em arquivo:

```

# registrar.py -- uso: python registrar.py COM3 degrau.csv
# Grava as linhas no arquivo; o que for digitado na janela vai para o kit.
import sys, serial, threading

porta, nome = sys.argv[1], sys.argv[2]
with serial.Serial(porta, 9600, timeout=2) as s, open(nome, "w") as f:
    def teclado():
        # comandos: A450, D1, D0
        for cmd in sys.stdin:
            s.write((cmd.strip() + "\r").encode("ascii"))
    threading.Thread(target=teclado, daemon=True).start()

    f.write("t_ms,temp_d,alvo_d,aquecedor,comutacoes\n")
    while True:
        linha = s.readline().decode("ascii", errors="replace").strip()
        if linha.count(",") == 4:
            # so linhas completas
            f.write(linha + "\n")
            f.flush()
            print(linha)

```

O programa junta o que o curso já tem: a base de 5 ms do R7, o conversor do R5, a histerese da aula 9 e a fila da aula 10. O buzzer é tocado **pelo laço**, consultando o Timer1, de propósito: ele é o instrumento que mede o bloqueio da Parte 2. No R7 ele era tocado por interrupção, e nenhum bloqueio o afetaria.

NOTA

3 Parte 1 — o quadro no fio

Tarefa 1. Calcule o divisor para 9600 bit/s e complete SPBRGH:SPBRG.

TAREFA · 1,0 pt

$n =$ _____ taxa real: _____ erro: _____

Com BUZZER em 0, grave, abra o terminal em 9600, 8N1, sem controle de fluxo, e confira as linhas chegando. Depois meça o pino TX (RC6) no osciloscópio, disparando na borda de descida.

	Previsto (P1)	Medido
Largura de um bit		
Duração de um quadro (10 bits)		

Identifique na tela o bit de início, os oito bits de dado e o bit de parada de um caractere. Qual caractere você capturou, e como sabe?

Tarefa 2. Sem regravar, mude o terminal para 19200 e depois para 4800. Volte a 9600.

TAREFA · 1,0 pt

Terminal em	O que aparece
19200	
4800	

Confere com a P2? O que “lixo” diz sobre a ligação, que “nada” não diria?

4 Parte 2 — quem espera desperdiça

Tarefa 3. Ligue CH3-6 e grave com BUZZER em 1 e BLOQUEANTE em 1. Ouça, e meça RC2 no osciloscópio com base de tempo de 10 ms/div, disparando pelo TX no canal 2. Depois grave com BLOQUEANTE em 0 e repita.

TAREFA · 2,0 pt

	Bloqueante	Fila
O que se ouve		
Lacuna na onda de RC2, por segundo		
Quadros no TX		

(a) Confere com a P3 e a P4? Explique a duração da lacuna pelo tamanho da linha.

(b) O TX mudou entre as duas versões? Onde está a diferença?

A 9600 bit/s, a linha leva 25 ms para sair, e isso não muda. O que muda é **quem espera**. Na versão bloqueante, o processador fica parado olhando o transmissor; na versão com fila, ele entrega a linha e segue. É o mesmo argumento do Timer1 no R6 e da nota no R7: o que o hardware faz sozinho não deveria custar processador.

CONCEITO

5 Parte 3 — o degrau, o dado que o R13 vai usar

Tarefa 4. Com BUZZER em 0, BLOQUEANTE em 0 e o aquecedor ainda frio, feche o terminal — a porta só aceita um programa por vez — e rode registrar.py para gravar degrau.csv. Ligue CH3-3, digite D1 na janela do script e deixe **dez minutos** registrando. Depois digite D0 e encerre com Ctrl+C.

TAREFA · 1,5 pt

Temperatura inicial (°C)
Temperatura após 10 min (°C)
Linhas gravadas
Taxa de aquecimento no primeiro minuto (°C/min)

(a) As linhas são exatamente uma por segundo? Como você verifica, só com o arquivo?

(b) A curva sobe em linha reta durante os dez minutos? O que isso diz sobre a planta?

Guarde `degrau.csv`. Ele é a entrada do R13: dele saem o modelo da planta, a sintonia feita em Python e o coeficiente que o curso vai “aprender” dos dados (aula 13). Um arquivo por grupo, com o nome do grupo.

ATENÇÃO

6 Parte 4 — o termostato, falando

Tarefa 5. Rode `registrar.py` gravando `termostato.csv`, com o alvo em 40,0 °C e o modo degrau desligado, por pelo menos dez minutos.

TAREFA · 1,5 pt

Temperatura máxima no regime (°C)
Temperatura mínima no regime (°C)
Amplitude (°C)
Comutações em 10 min
Comutações por hora (extrapolado)

(a) A amplitude medida é maior que a largura da histerese (1,0 °C)? Por quê?

(b) Esta tabela é a das duas medidas da aula 9. Por que ela não poderia ser preenchida só com o display?

Tarefa 6. Com o registro da Tarefa 5 rodando, digite A450 na janela do script. Depois A900 e A45.

TAREFA · 1,0 pt

Comando	O que aconteceu com <code>alvo_d</code>
A450	
A900	
A45	

(a) Por que o programa recusa A900 e A45? O que aconteceria sem essa verificação?

(b) Se o programa ficasse muito tempo sem ler RCREG, o que aconteceria com a recepção, e como o programa se recupera?

7 Armadilhas frequentes

Sintoma	Causa provável
Nada chega ao terminal	Alguma chave de CH5 em ON (relés em RC6/RC7), CH4-6/CH4-8 em OFF, ou cabo <i>null-modem</i>
Caracteres embaralhados	Taxa diferente entre o kit e o terminal
A serial funciona e depois emudece	0ERR: a recepção parou e ninguém religou CREN
Programa preso ao ligar	TXIE ligado com a fila vazia, ou RCIF testado sem ler RCREG
registrar.py não abre a porta	O terminal ainda está com a porta aberta: só um programa por vez
Temperatura absurda	CH1-1, CH1-5 ou CH1-6 em ON, disputando RA0

8 Entrega

As tabelas das Tarefas 1 a 6, e os arquivos `degrau.csv` e `termostato.csv`, com o nome do grupo.

TAREFA

Responda também:

(a) Que parte do programa roda no tratamento, e que parte no laço? Onde entra a espera que sobrou?

(b) Por que texto e não binário, se o binário ocuparia um terço?

9 Se sobrar tempo

E1 — o gráfico. Em Python, com `pandas` e `matplotlib`, trace `temp_d / 10` e `aquecedor` contra `t_ms / 60000` para os dois arquivos.

OPCIONAL

```
import pandas as pd, matplotlib.pyplot as plt
d = pd.read_csv("degrau.csv")
plt.plot(d.t_ms / 60000, d.temp_d / 10)
plt.xlabel("min"); plt.ylabel("°C"); plt.show()
```

E2 — o custo do `printf`. Troque `enviar_u32` por `printf` com `putch` redirecionado para `uart_byte`, e compare o tamanho do programa no arquivo de mapa.

OPCIONAL

E3 — 115200. Calcule o divisor para 115200 bit/s, grave e confira no terminal. Qual o erro? Sobra margem para o computador?

OPCIONAL

No R11: guardar o alvo e os parâmetros na memória que sobrevive a desligar a placa. No R13, os arquivos de hoje viram modelo, sintonia e os coeficientes que descem para o chip.

NOTA